

Stochastic Gradient Descent

Why it can be cheaper than full gradient descent, and what the mathematics says

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education
University of Oslo

Week 39 supplement

The finite-sum problem

For n observations and parameters $\boldsymbol{\theta} \in \mathbb{R}^p$, write the empirical risk as a mean,

$$F(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n f_i(\boldsymbol{\theta}), \quad \nabla F(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\boldsymbol{\theta}).$$

Full gradient descent

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla F(\boldsymbol{\theta}_t).$$

One update reads all n observations.

The comparison must use a common budget: examples processed, floating-point operations, or wall time.

Minibatch SGD

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{g}_t, \quad \mathbf{g}_t = \frac{1}{M} \sum_{i \in B_t} \nabla f_i(\boldsymbol{\theta}_t).$$

One update reads only $M \ll n$ observations.

Cost per update and cost per epoch

For a dense linear model, evaluating a loss gradient requires matrix-vector products. The leading operation counts are

$$\text{full gradient update} \approx 4np, \quad \text{minibatch update} \approx 4Mp.$$

An epoch contains n/M minibatch updates, hence

$$\frac{n}{M} (4Mp) = 4np.$$

The important comparison

One epoch of minibatch SGD costs approximately one full-gradient update, but SGD changes the parameters n/M times during that epoch.

Those extra updates help most early in training. Near the minimum, gradient noise limits their value.

A million-sample calculation

Take $n = 10^6$, $p = 100$, minibatch size $M = 64$, and 32-bit floating-point data.

	Full gradient	Minibatch SGD
Examples per update	10^6	64
Leading operations per update	4×10^8	2.56×10^4
Updates per epoch	1	15 625
Leading operations per epoch	4×10^8	4×10^8

If full gradient descent needs 30 passes to reach the required validation loss, it uses about

$$30(4np) = 1.2 \times 10^{10} \text{ leading operations.}$$

If SGD reaches the same validation loss after 4 passes, it uses about

$$4(4np) = 1.6 \times 10^9,$$

which is 7.5 times less work. The assumed pass counts must be measured for the problem at hand.

Least squares makes the operation count explicit

For

$$F(\boldsymbol{\theta}) = \frac{1}{2n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2, \quad \nabla F = \frac{1}{n} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}),$$

a dense full gradient performs two matrix-vector products with $\mathbf{X} \in \mathbb{R}^{n \times p}$. For a minibatch B ,

$$\mathbf{g}_B = \frac{1}{M} \mathbf{X}_B^T (\mathbf{X}_B \boldsymbol{\theta} - \mathbf{y}_B), \quad \mathbf{X}_B \in \mathbb{R}^{M \times p}.$$

- ▶ Full gradient: approximately $4np$ multiply-add operations and a scan through the full design matrix.
- ▶ Minibatch gradient: approximately $4Mp$ operations and a much smaller working set.
- ▶ A minibatch may remain in fast memory, while the full matrix may require repeated memory traffic.

The wall-time advantage can therefore exceed the elementary arithmetic comparison.

A worked one-dimensional example

Let

$$f_i(\theta) = \frac{1}{2}(\theta - a_i)^2, \quad F(\theta) = \frac{1}{n} \sum_{i=1}^n f_i(\theta).$$

Then $\theta^* = \bar{a}$ and

$$F'(\theta) = \theta - \bar{a}.$$

Full gradient descent with $\eta = 1$ finds the empirical minimizer in one update,

$$\theta_1 = \theta_0 - (\theta_0 - \bar{a}) = \bar{a},$$

but computing $\bar{a}$ reads all n values. SGD samples i_t and uses

$$\theta_{t+1} = (1 - \eta_t)\theta_t + \eta_t a_{i_t}.$$

With $\eta_t = 1/(t+1)$, θ_t is the running sample mean. After $t \ll n$ observations,

$$\mathbb{E}[\theta_t] = \bar{a}, \quad \text{Var}(\theta_t) \approx \frac{s_a^2}{t}.$$

SGD obtains a useful approximation without first scanning the entire data set.

Unbiasedness of the minibatch gradient

Sample indices $l_1, \dots, l_M$ independently and uniformly from $\{1, \dots, n\}$. Define

$$\mathbf{g}_B(\boldsymbol{\theta}) = \frac{1}{M} \sum_{m=1}^M \nabla f_{l_m}(\boldsymbol{\theta}).$$

Conditioned on the current iterate $\boldsymbol{\theta}$,

$$\begin{aligned} \mathbb{E}[\mathbf{g}_B \mid \boldsymbol{\theta}] &= \frac{1}{M} \sum_{m=1}^M \mathbb{E}[\nabla f_{l_m}(\boldsymbol{\theta}) \mid \boldsymbol{\theta}] \\ &= \frac{1}{M} \sum_{m=1}^M \frac{1}{n} \sum_{i=1}^n \nabla f_i(\boldsymbol{\theta}) \\ &= \nabla F(\boldsymbol{\theta}). \end{aligned}$$

Thus the stochastic direction is correct on average. An individual update need not decrease F .

Conditional expectation matters

The batch must be chosen independently of the current gradient noise, given the current iterate.

Variance and the role of minibatch size

Let

$$\boldsymbol{\xi}_i = \nabla f_i(\boldsymbol{\theta}) - \nabla F(\boldsymbol{\theta}), \quad \mathbb{E}[\boldsymbol{\xi}_i \mid \boldsymbol{\theta}] = 0.$$

For independent sampling with replacement,

$$\mathbb{E}[\|\mathbf{g}_B - \nabla F\|^2 \mid \boldsymbol{\theta}] = \frac{1}{M} \mathbb{E}[\|\boldsymbol{\xi}_i\|^2 \mid \boldsymbol{\theta}] = \frac{\sigma_g^2(\boldsymbol{\theta})}{M}.$$

Hence the standard deviation falls as $1/\sqrt{M}$, not as $1/M$.

For a batch drawn uniformly without replacement, component j satisfies

$$\text{Var}(g_{B,j} \mid \boldsymbol{\theta}) = \frac{s_j^2}{M} \frac{n - M}{n - 1}.$$

The finite-population correction makes the variance exactly zero at $M = n$.

Expected progress in one SGD update

Assume that F is L -smooth:

$$F(\mathbf{y}) \leq F(\mathbf{x}) + \langle \nabla F(\mathbf{x}), \mathbf{y} - \mathbf{x} \rangle + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2.$$

Insert $\mathbf{y} = \boldsymbol{\theta}_t - \eta \mathbf{g}_t$, use unbiasedness, and suppose $\mathbb{E}[\|\mathbf{g}_t - \nabla F\|^2 \mid \boldsymbol{\theta}_t] \leq \sigma_g^2/M$. Then

$$\mathbb{E}[F(\boldsymbol{\theta}_{t+1}) \mid \boldsymbol{\theta}_t] \leq F(\boldsymbol{\theta}_t) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla F(\boldsymbol{\theta}_t)\|^2 + \frac{L\eta^2\sigma_g^2}{2M}.$$

- ▶ The middle term is deterministic progress.
- ▶ The last term is the price of stochastic gradients.
- ▶ Larger M reduces noise, while smaller M gives more updates per pass.

Strong convexity and convergence rates

Suppose F is μ -strongly convex and L -smooth, with condition number $\kappa = L/\mu$. For full gradient descent with a suitable fixed step,

$$F(\boldsymbol{\theta}_t) - F^* \leq \left(1 - \frac{1}{\kappa}\right)^t (F(\boldsymbol{\theta}_0) - F^*).$$

Reaching optimization error ε therefore takes

$$t_{\text{GD}} = O\left(\kappa \log \frac{F(\boldsymbol{\theta}_0) - F^*}{\varepsilon}\right)$$

full passes through the data.

With a decaying step $\eta_t \asymp 1/(\mu t)$, a standard SGD bound is

$$\mathbb{E}[F(\boldsymbol{\theta}_t) - F^*] = O\left(\frac{\sigma_g^2}{\mu M t}\right),$$

after the transient has decayed. SGD is slower in update count, but every update costs only M/n of a full pass.

Why the target accuracy changes the winner

Optimization acts on a finite sample. For many regular parametric models, the statistical estimation error scales as

$$\varepsilon_{\text{stat}} = O\left(\frac{p}{n}\right)$$

up to noise and conditioning factors. Reducing the training objective far below this scale may improve the empirical fit without improving prediction on new data.

Target	Likely computational choice
Moderate validation accuracy	Small or medium minibatches often reach it in a few passes
Very accurate empirical minimizer	Full gradients or variance-reduced methods become competitive
Streaming or changing data	SGD can update before a complete data set exists
Small data set	Full gradient descent may be simpler and faster

The statement “SGD is faster” always requires an accuracy target and a cost measure.

The constant-step noise floor

Consider one eigendirection near a quadratic minimum,

$$e_{t+1} = (1 - \eta\lambda)e_t - \eta\xi_t, \quad \mathbb{E}[\xi_t] = 0, \quad \text{Var}(\xi_t) = \frac{\sigma_g^2}{M}.$$

The second moment satisfies

$$\mathbb{E}[e_{t+1}^2] = (1 - \eta\lambda)^2 \mathbb{E}[e_t^2] + \eta^2 \frac{\sigma_g^2}{M}.$$

If $0 < \eta\lambda < 2$, the stationary variance is

$$\text{Var}(e_\infty) = \frac{\eta^2 \sigma_g^2 / M}{1 - (1 - \eta\lambda)^2} = \frac{\eta \sigma_g^2}{M\lambda(2 - \eta\lambda)} \approx \frac{\eta \sigma_g^2}{2M\lambda}.$$

A constant step produces a cloud around the minimizer. More updates do not remove this floor.

Decaying learning rates

The Robbins-Monro conditions are

$$\sum_{t=0}^{\infty} \eta_t = \infty, \quad \sum_{t=0}^{\infty} \eta_t^2 < \infty.$$

The first condition prevents the algorithm from stopping too early. The second limits the accumulated gradient noise.

Common choices include

$$\eta_t = \frac{\eta_0 t_0}{t + t_0}, \quad \eta_t = \frac{\eta_0}{(1 + t/t_0)^\alpha}, \quad \frac{1}{2} < \alpha \leq 1.$$

- ▶ A long warm phase can move rapidly toward a useful region.
- ▶ Later decay shrinks the stationary cloud.
- ▶ The schedule uses update count t , so changing M changes the number of updates per epoch.

The batch-size trade-off

Increasing M changes statistics and hardware utilization at the same time.

	Small batch	Large batch
Gradient variance	Larger	Smaller, approximately $1/M$
Updates per epoch	Many	Few
Parallel efficiency	May underuse GPU or vector units	Usually better until saturation
Memory use	Small	Larger
Late-stage accuracy	Needs decay or averaging	More stable at fixed step

A simple wall-time model is

$$T_{\text{update}}(M) \approx T_{\text{overhead}} + \frac{M}{R(M)},$$

where $R(M)$ is the achieved processing rate. The best M often lies near hardware saturation, not at $M = 1$.

Data passes, updates, and wall time

Three plots can tell different stories:

- ▶ **Loss versus update count:** favours cheap noisy updates because GD has only one update per epoch.
- ▶ **Loss versus epochs:** compares the amount of data processed.
- ▶ **Loss versus wall time:** includes memory traffic, parallelism, compilation and communication.

For a fair experiment:

1. Use the same training and validation split.
2. Tune learning rates separately for GD and SGD.
3. Compare at equal validation performance, not only equal iteration count.
4. Report several random seeds and uncertainty bands.

An efficiency claim without a common horizontal axis is incomplete.

Example protocol for the course

Use a synthetic least-squares problem with $n = 100\,000$, $p = 50$, correlated standardized features and controlled noise.

Method	Batch size	Learning rate	Recorded quantities
GD	n	tuned constant	train loss, validation loss, time
SGD	1	decaying	same quantities
Minibatch SGD	32, 128, 512	tuned and decaying	same quantities

Plot validation loss against

updates, examples processed, estimated operations, wall time.

Repeat for several condition numbers. Ill conditioning slows full gradient descent and can also require preconditioning or momentum for SGD.

A practical SGD loop

```
theta = np.zeros(p)
t = 0
for epoch in range(max_epochs):
    for batch in shuffled_batches(n, batch_size, rng):
        t += 1
        g = gradient(theta, X[batch], y[batch])
        eta = eta0 * t0 / (t + t0)
        theta -= eta * g

    val = cost(theta, X_val, y_val)
    if early_stopping(val):
        break
```

- ▶ Shuffle once per epoch and keep the final smaller batch.
- ▶ Average the batch loss or adjust the learning rate if gradients are summed.
- ▶ Standardize features so one coordinate does not dictate the stable step size.
- ▶ Use a validation objective for stopping.

When SGD is not more efficient

Full gradient descent can win when

- ▶ n is small enough that the full data set fits comfortably in fast memory,
- ▶ the required solution is very accurate,
- ▶ stochastic gradients have extremely high variance,
- ▶ large matrix operations use the hardware much better than small batches,
- ▶ the objective permits caching or a cheap sufficient statistic.

For least squares, for example, forming $X^T X$ and $X^T \mathbf{y}$ once may be attractive when $p \ll n$ and the $p \times p$ matrix fits in memory. The cost and numerical stability of that alternative must also be considered.

A precise conclusion

SGD is computationally efficient when a useful solution requires only a few data passes and each minibatch update provides enough directional information to make progress.

Summary

- ▶ A full gradient update costs $O(np)$ for a dense linear model; a minibatch update costs $O(Mp)$.
- ▶ One SGD epoch and one full-gradient update have similar leading arithmetic cost.
- ▶ SGD may make thousands of parameter updates during that same pass through the data.
- ▶ The minibatch gradient is unbiased, with variance approximately proportional to $1/M$.
- ▶ Constant learning rates create a noise floor of order η/M near a quadratic minimum.
- ▶ Decaying rates, iterate averaging and validation-based stopping convert cheap noisy updates into useful accuracy.
- ▶ Efficiency must be reported against examples processed, operations, or wall time at the same validation performance.